Understanding Key-Value Format in Flutter

a. What is a Key-Value Pair?

- In Flutter, widget properties are defined using key-value pairs.
- Key → Represents a property (e.g., style, color).
- Value → Defines how the property behaves (e.g., Colors.blue, TextStyle).
- For example, in the Text widget:

```
Text(
  'Hello, Flutter!',      // This is a value (text being
displayed)
  style: TextStyle(       // 'style' is a key
    fontSize: 20,       // 'fontSize' is a key, and '20' is the
value
    color: Colors.blue,   // 'color' is a key, and 'Colors.blue'
is the value
  ),
);
```

b. Why Use Key-Value Pairs?

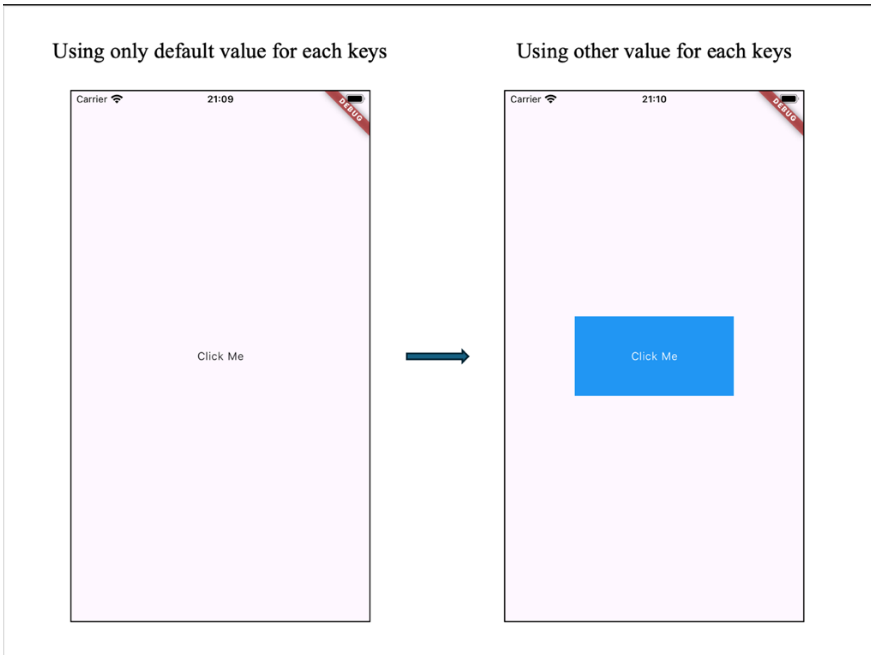
- *Customizes widget properties* → Easily control UI appearance & behavior.
- *Overrides default values* → Adjust built-in properties like color, size, and alignment.
- *Improves readability* → Named parameters make Flutter code clearer.

c. When Do You Need to Change the Key's Default Value?

- If no value is provided, Flutter applies a default value.
- You override the default when you need a different behavior.
- This is an example for changing default values :

```
Container(
  width: 200,
  height: 100,
  color: Colors.blue, // This sets the background color to
blue
  child: Center(
    child: Text(
      'Click Me',
      style: TextStyle(color: Colors.white), // Text color
is white
    ),
  ),
)
```

- Explanation:
  - By default, a Container has no background color.
  - Setting color: Colors.blue changes the background to blue.
  - Inside, Text('Click Me') is centered and displayed in white.



d. Using Key-Value Pairs in Flutter vs. JSON
- Both Flutter & JSON use key-value pairs, but they serve different purposes.
- Flutter → Defines how an app looks (UI).

```
Text(
  "Hello, World!",
  textAlign: TextAlign.center,
)
```

- JSON → Stores and transfers data.

```
{
  "text": "Hello, World!",
  "textAlign": "center"
}
```

- Key Difference :

Feature	Flutter Code	JSON
Purpose	UI strusture & logic	Data storage & transfer
Execution	Runs in an app	Used to pass data
Example	`Text('Hello')`	`{ "text": "Hello" }`

Why Widgets Are Nested in Flutter

a. What is Widget Nesting?
- Nesting widgets means placing one widget inside another.
- This allows you to combine UI elements into more complex designs.
- Every Flutter UI is built using a hierarchy of nested widgets.
- For example, a Text widget inside a Button :

```
ElevatedButton(
```